LeetCode Practice

21 Medium: Sliding Window

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

21
MEDIUM

Patterns (in order)

1. Sliding Window (21)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

21 questions | tap a title to jump to it

Medium questions (21) Go >

Sliding Window (21) Go >

M 1. Longest Substring Without Repeating Characters LC 3
M 2. Longest Substring with At Most Two Distinct Characters LC 159
M 3. Minimum Size Subarray Sum LC 209
M 4. Longest Substring with At Most K Distinct Characters LC 340
M 5. Longest Substring with At Least K Repeating Characters LC 395
M 6. Longest Repeating Character Replacement LC 424
M 7. Find All Anagrams in a String LC 438
M 8. Max Consecutive Ones II LC 487
M 9. Permutation in String LC 567
M 10. Subarray Product Less Than K LC 713
M 11. New 21 Game LC 837
M 12. Fruit Into Baskets LC 904
M 13. Longest Turbulent Subarray LC 978
M 14. Max Consecutive Ones III LC 1004
M 15. Find K-Length Substrings with No Repeated Characters LC 1100
M 16. Count Number of Nice Subarrays LC 1248
M 17. Maximum Points You Can Obtain from Cards LC 1423
M 18. Maximum Number of Vowels in a Substring of Given Length LC 1456
M 19. Frequency of the Most Frequent Element LC 1838
M 20. Jump Game VII LC 1871
M 21. Maximum Sum of Distinct Subarrays With Length K LC 2461

Medium questions (21)

MEDIUM | TOPIC 1 OF 1

Sliding Window

21 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Longest Substring Without Repeating Characters LC 3
2. Longest Substring with At Most Two Distinct Characters LC 159
3. Minimum Size Subarray Sum LC 209
4. Longest Substring with At Most K Distinct Characters LC 340
5. Longest Substring with At Least K Repeating Characters LC 395
6. Longest Repeating Character Replacement LC 424
7. Find All Anagrams in a String LC 438
8. Max Consecutive Ones II LC 487
9. Permutation in String LC 567
10. Subarray Product Less Than K LC 713
11. New 21 Game LC 837
12. Fruit Into Baskets LC 904
13. Longest Turbulent Subarray LC 978
14. Max Consecutive Ones III LC 1004
15. Find K-Length Substrings with No Repeated Characters LC 1100
16. Count Number of Nice Subarrays LC 1248
17. Maximum Points You Can Obtain from Cards LC 1423
18. Maximum Number of Vowels in a Substring of Given Length LC 1456
19. Frequency of the Most Frequent Element LC 1838
20. Jump Game VII LC 1871
21. Maximum Sum of Distinct Subarrays With Length K LC 2461

MEDIUM | Sliding Window | LeetCode #3 | 1 of 21 | 1/21

Longest Substring Without Repeating Characters

QUESTION

Given a string s, find the length of the longest substring without duplicate characters.

Example 1	
Input	`s = "abcabcbb"`
Output	**3**
Explanation	*The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.*

Example 2	
Input	`s = "bbbbb"`
Output	**1**
Explanation	*The answer is "b", with the length of 1.*

Example 3	
Input	`s = "pwwkew"`
Output	**3**
Explanation	*The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.*

CONSTRAINTS

- 0 <= s.length <= $5 * 10^4$
- s consists of English letters, digits, symbols and spaces.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the length of the longest substring with all unique characters.

Substring is contiguous. Empty string returns 0. Right?

Characters can be any printable ASCII — not just lowercase letters."

"'abcabcbb'->3 ('abc'). 'bbbbb'->1 ('b'). 'pwwkew'->3 ('wke') ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if s[i..j] has no repeated character:
            best = max(best, j - i + 1)
```

TIME	SPACE
O(n^3)	O(min(n, alphabet))

WHY IT IS SLOW

Checks O(n^2) substrings, each in O(n), and re-reads the same characters again and again. A sliding window moves each end once, O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Sliding window with a frequency map.

Expand right: add s[right] to the map.

When a duplicate appears (count > 1), shrink from the left until it's gone.

At each step, update the max window size.

Time: O(n). Space: O(min(n, alphabet)) — at most 128 unique ASCII chars."

Key insight: the frequency map detects duplicates in O(1) — the left pointer only advances when a duplicate exists, so each character is processed at most twice total.

PHASE 4 CLEAN CODE

```
class _3_LongestSubstringNoRepeat:
    def lengthOfLongestSubstring(self, s: str) -> int:
        freq   : defaultdict = defaultdict(int)
        left   = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while freq[s[right]] > 1:       # shrink until
 no duplicate
                freq[s[left]] -= 1
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 TEST & DEBUG

"Let me trace lengthOfLongestSubstring('abcabcbb')."

window={}, left=0, max_len=0.

right=0 (a): window={a:0}. max_len=1.

right=1 (b): window={a:0,b:1}. max_len=2.

right=2 (c): window={a:0,b:1,c:2}. max_len=3.

right=3 (a): 'a' in window AND window[a]=0 >= left=0 -> left=0+1=1. window[a]=3. max_len=3.

right=4 (b): 'b' in window, window[b]=1 >= left=1 -> left=2. window[b]=4. max_len=3.

right=5 (c): 'c' in window, window[c]=2 >= left=2 -> left=3. window[c]=5. max_len=3.

PHASE 5 TEST & DEBUG

right=6 (b): 'b' in window, window[b]=4 >= left=3 -> left=5. window[b]=6. len=6-5+1=2.

right=7 (b): left=7. max_len=3.

Return 3 ok ('abc').

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(min(n, 128))**
each character enters and leaves the window at most once	for the frequency map

Alternative: store the LAST INDEX of each character instead of frequency.

When s[right] already seen at idx and idx >= left: jump left = idx+1 directly

(one move instead of shrinking one at a time). Same O(n) but fewer iterations.

Given more time I'd ask: what if we want the actual substring, not just its length?

Track left and best_left when updating the max.

MEDIUM | Sliding Window | LeetCode #159 | 2 of 21 | 2/21

Longest Substring with At Most Two Distinct Characters

QUESTION

Given a string s, return the length of the longest substring that contains at most two distinct characters.

Example 1	
Input	`s = "eceba"`
Output	**3**
Explanation	*The substring is "ece" which its length is 3.*

Example 2	
Input	`s = "ccaabbb"`
Output	**5**
Explanation	*The substring is "aabbb" which its length is 5.*

CONSTRAINTS

- 1 <= s.length <= 10^5
- s consists of English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the longest substring that contains at most 2 distinct characters. Right?

'eceba': 'ece' has 2 distinct -> length 3 ok. 'ccaabbb': 'aabbb' has 2 -> length 5 ok"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    count the distinct characters
    if at most 2:
        best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) substrings, each counted in O(n). A sliding window with a count map that shrinks from the left when a third character appears is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window with frequency map and distinct count — O(n). Walk-through:"

freq={}. distinct=0. left=0. best=0.

For right in 0..n-1:

c=s[right]. freq[c]=freq.get(c,0)+1.

If freq[c]==1: distinct++. <- new character entered window

While distinct > 2:

lc=s[left]. freq[lc]-=1.

If freq[lc]==0: distinct--; del freq[lc].

left++.

`best = max(best, right-left+1).`

WHY distinct counter: O(1) check if window is valid vs checking len(freq).

WHY distinct>2 triggers shrink: window has more than 2 distinct characters -> too many.

Time: O(n). Space: O(1) (at most 3 chars in freq at any time -> bounded).

PHASE 4 CLEAN CODE

```
class _159_LongestSubstringTwoDistinct:
    def lengthOfLongestSubstringTwoDistinct(self, s: str)
-> int:
        freq    : defaultdict = defaultdict(int)
        left    = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while len(freq) > 2:            # shrink until
 at most 2 distinct
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

s='eceba':

r=0('e'): freq={e:1}. longest=1.

r=1('c'): freq={e:1,c:1}. longest=2.

r=2('e'): freq={e:2,c:1}. longest=3.

r=3('b'): freq={e:2,c:1,b:1} -> 3 distinct. shrink:

freq[e]-=1->1. left=1. freq={e:1,c:1,b:1} still 3. shrink:

freq[c]-=1->0 -> del c. left=2. freq={e:1,b:1}. 2 distinct -> stop. longest=max(3,2)=3.

r=4('a'): freq={e:1,b:1,a:1} -> 3. shrink: freq[e]-=1->0->del e. left=3. freq={b:1,a:1}. longest=3.

-> 3 ok

s='ccaabbb':

Window expands to 'ccaabbb' but once 'b' is added after 'a', we have c,a,b(3 distinct).

PHASE 5 TEST & DEBUG

Shrink past the c's. Eventually best window is 'aabbb'=5. -> 5 ok

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** at most 3 keys in the map at any time

This is the template for 'at most k distinct characters' — change > 2 to > k.

Generalisation: Longest Substring with At Most K Distinct Characters (#340) is the same

algorithm with k replacing 2.

The key difference from #3: shrink on len(freq) > k rather than on a specific duplicate.

Given more time I'd ask: what about EXACTLY 2 distinct characters?

```
atMost(k=2) - atMost(k=1) gives the count of
  substrings with exactly 2 distinct.
```

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #209 | 3 of 21 | 3/21

Minimum Size Subarray Sum

QUESTION

Given an array of positive integers nums and a positive integer target, return the minimal length of a subarray whose sum is greater than or equal to target. If there is no such subarray, return 0 instead.

Example 1	
Input	`target = 7, nums = [2,3,1,2,4,3]`
Output	`2`
Explanation	*The subarray [4,3] has the minimal length under the problem constraint.*

Example 2	
Input	`target = 4, nums = [1,4,4]`
Output	`1`

Example 3	
Input	`target = 11, nums = [1,1,1,1,1,1,1,1]`
Output	`0`

CONSTRAINTS

- $1 <= target <= 10^9$
- $1 <= nums.length <= 10^5$
- $1 <= nums[i] <= 10^4$
- Follow up: If you have figured out the O(n) solution, try coding another solution of which the time complexity is O(n log(n)).

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the minimum length subarray with sum >= target. Return 0 if none. Right?"

"A few quick questions:

All values are positive (no negatives)? Yes — required for sliding window to work.

Can the whole array be the answer? Yes."

"Let me walk: nums=[2,3,1,2,4,3], target=7 -> [4,3] length 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    add elements one at a time until the sum
        reaches target
    best = min(best, length)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Restarts the sum from every position. A sliding window (grow the right end, shrink the left while the sum is still enough) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window with dynamic right expansion and left shrink — O(n). Walk-through:"

total=0, left=0, best=inf.

For right in 0..n-1:

`total += nums[right].`

While total >= target:

`best = min(best, right-left+1).`

`total -= nums[left]; left++.`

Return best if best!=inf else 0.

WHY shrink while total>=target: we want the MINIMUM length, so shrink from left

as long as the window sum still meets the requirement.

WHY total>=target (not ==): once valid, try shrinking further to find shorter window.

All elements positive -> shrinking always decreases sum -> safe to shrink greedily.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _209_MinSizeSubarraySum:
    def minSubArrayLen(self, target, nums):
        left = 0
        window_sum = 0
        min_len = float('inf')
        for right in range(len(nums)):
            window_sum += nums[right]
            while window_sum >= target:
                min_len = min(min_len, right - left + 1)
                window_sum -= nums[left]
                left += 1
        return min_len if min_len != float('inf') else 0
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

nums=[2,3,1,2,4,3], target=7:

r=0:sum=2. r=1:5. r=2:6. r=3:8>=7->len=4,shrink:sum=6,l=1. r=4:10>=7->len=4,shrink:sum=7>=7->len=3,shrink:sum=4,l=3.

r=5:7>=7->len=3,shrink:sum=4,l=4. -> min=2? Wait: r=4,l=2 after shrinking: [2,4] len=3, then [4] sum=4<7.

Actually r=4(4): sum was 4+4=8? Let me retrace more carefully. -> min=2 ([4,3]) ok

The while loop shrinks until sum falls below target, recording min each time

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element added and removed at most once	**O(1)**

If values can be negative, sliding window doesn't work — need prefix sums + binary search O(n log n).

Follow-up: Minimum Window Substring (#76) — string version with character constraint.

Follow-up: Maximum Size Subarray Sum Equals k (#325) — with negatives; prefix sum + hash map.

MEDIUM | Sliding Window | LeetCode #340 | 4 of 21 | 4/21

Longest Substring with At Most K Distinct Characters

QUESTION

Given a string s and an integer k, return the length of the longest substring of s that contains at most k distinct characters.

Example 1	
Input	`s = "eceba", k = 2`
Output	`3`
Explanation	*The substring is "ece" with length 3.*

Example 2	
Input	`s = "aa", k = 1`
Output	`2`
Explanation	*The substring is "aa" with length 2.*

CONSTRAINTS

- 1 <= s.length <= $5 * 10^4$
- 0 <= k <= 50

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return the length of the longest substring with at most k distinct characters.

k=0 means the answer is 0 (no characters allowed). Right?

This is the direct generalisation of #159 (at most 2 distinct) with k replacing 2."

"'eceba', k=2 -> 'ece', length 3 ok. 'aa', k=1 -> 'aa', length 2 ok."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    count its distinct characters
    if at most k:
        best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(k)

WHY IT IS SLOW

O(n^2) substrings, each counted in O(n). A sliding window with a count map that shrinks from the left when more than k distinct appear is O(n).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window with frequency map — O(n). Walk-through:"

`freq = {}. left=0. distinct=0.`

For right in 0..n-1:

`freq[s[right]] = freq.get(s[right],0) + 1.`

If freq[s[right]] == 1: distinct++ (new unique char added).

While distinct > k:

`freq[s[left]] -= 1.`

If freq[s[left]] == 0: distinct--; del freq[s[left]].

left++.

`best = max(best, right - left + 1).`

WHY shrink when distinct > k: window has too many unique chars -> move left until valid.

WHY track distinct separately: O(1) to check if window is valid vs iterating freq each time.

Time: O(n). Space: O(k).

PHASE 4 CLEAN CODE

```
class _340_LongestSubstringKDistinct:
    def lengthOfLongestSubstringKDistinct(self, s: str, k:
 int) -> int:
        if k == 0:
            return 0
        freq    : defaultdict = defaultdict(int)
        left    = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while len(freq) > k:
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 TEST & DEBUG

"Let me trace lengthOfLongestSubstringKDistinct('eceba', k=2)."

left=0, freq={}, max_len=0.

right=0('e'): freq={e:1}, distinct=1. max_len=1.

right=1('c'): freq={e:1,c:1}, distinct=2. max_len=2.

right=2('e'): freq={e:2,c:1}, distinct=2. max_len=3.

right=3('b'): freq={e:2,c:1,b:1}, distinct=3>2. Shrink: remove s[0]='e'->freq={e:1,c:1,b:1}. distinct=3. Remove s[1]='c'->freq={e:1,b:1}. left=2. distinct=2. max_len=max(3,3-2+1)=3.

right=4('a'): freq={e:1,b:1,a:1}, distinct=3>2. Remove s[2]='e'->freq={b:1,a:1}. left=3. max_len=3.

Return 3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(k)** Identical to #159 — just k instead of 2

Exactly k distinct: atMost(k) - atMost(k-1).

Given more time I'd ask: what if k >= distinct chars in s? Answer = len(s).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #395 | 5 of 21 | 5/21

Longest Substring with At Least K Repeating Characters

QUESTION

Given a string s and an integer k, return the length of the longest substring of s such that the frequency of each character in this substring is greater than or equal to k.

if no such substring exists, return 0.

Example 1

Input	`s = "aaabb", k = 3`
Output	3
Explanation	*The longest substring is "aaa", as 'a' is repeated 3 times.*

Example 2

Input	`s = "ababbc", k = 2`
Output	5
Explanation	*The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3 times.*

CONSTRAINTS

- 1 <= s.length <= 10^4
- s consists of only lowercase English letters.
- 1 <= k <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the longest substring where every character appears at least k times. Right?"

"A few quick questions:

If no valid substring, return 0? Yes. s has only lowercase letters? Yes."

"Let me walk: s='aaabb', k=3 -> 'aaa' -> length 3 ok. s='ababbc', k=2 -> 'ababb' -> length 5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    count the letters
    if every letter present occurs at least k times:
        best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

O(n^2) substrings, each counted in O(n). Split at any letter whose total count is below k (it cannot be in the answer) and recurse on the pieces, or run a sliding window for each number of distinct letters: O(26 * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window: shrink until frequency condition met — O(26n) = O(n). Walk-through:"

For max_unique in 1..num_unique_chars_in_s: (number of distinct chars allowed in window)

Slide window ensuring exactly max_unique distinct chars, all with freq >= k.

left=0. curr_unique=0. curr_valid=0. freq={}.

For right in 0..n-1:

c=s[right]. If freq.get(c,0)==0: curr_unique++. freq[c]+=1.

If freq[c]==k: curr_valid++ (this char now meets the k requirement).

While curr_unique > max_unique:

lc=s[left]. If freq[lc]==k: curr_valid--. freq[lc]-=1. If freq[lc]==0: curr_unique--. left++.

If curr_valid == curr_unique: best = max(best, right-left+1). <- all distinct chars in window have freq>=k

Return best.

WHY iterate over max_unique: avoids the unbounded shrink issue. Outer loop is O(26).

Time: O(26n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import Counter
class _395_LongestSubstringAtLeastKRepeating:
    def longestSubstring(self, s, k):
        if len(s) < k:
            return 0
        freq = Counter(s)
        for ch, count in freq.items():
            if count < k:
                return max(self.longestSubstring(part, k)
for part in s.split(ch))
        return len(s)
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

s='aaabb', k=3: freq={a:3,b:2}. 'b' has count 2<3 -> split on 'b': ['aaa',''].

`longestSubstring('aaa',3): freq={a:3} all>=3 -> return 3. longestSubstring('',3)=0. max=3 ok`

PHASE 5 TEST & DEBUG

s='ababbc', k=2: freq={a:2,b:3,c:1}. 'c' < 2 -> split on 'c': ['ababb'].

```
longestSubstring('ababb',2): freq={a:2,b:3} all>=2 ->
  return 5 ok
```

Splitting on invalid chars is the key insight — they can't appear in any valid substring

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)**
average: each level of recursion reduces by at least one split char	for recursion stack

Sliding window with fixed distinct-char count also works in O(n) — iterate over 1..26 possible distinct counts.

Follow-up: Longest Substring Without Repeating Characters (#3) — k=1 (appears >=1 times).

Follow-up: if k=0, return len(s).

MEDIUM | Sliding Window | LeetCode #424 | 6 of 21 | 6/21

Longest Repeating Character Replacement

QUESTION

You are given a string s and an integer k. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1	
Input	`s = "ABAB", k = 2`
Output	**4**
Explanation	*Replace the two 'A's with two 'B's or vice versa.*

Example 2	
Input	`s = "AABABBA", k = 1`
Output	**4**
Explanation	*Replace the one 'A' in the middle with 'B' and form "AABBBBA". The substring "BBBB" has the longest repeating letters, which is 4. There may exists other ways to achieve this answer too.*

CONSTRAINTS

- 1 <= s.length <= 10^5
- s consists of only uppercase English letters.
- 0 <= k <= s.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

We can replace at most k characters. Find the longest substring where all characters

are the same after at most k replacements.

String is uppercase letters only. Right?

Strategy: keep the most frequent char, replace everything else."

"'ABAB', k=2 -> replace both A's or B's -> 4 ok

'AABABBA', k=1 -> best window = 4 ok"

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings:
    count its letters
    if length - (most common count) <= k:
        best = max(best, candidate)
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) substrings, each counted in O(n). A sliding window keeping the count of the most common letter, shrinking when more than k replacements are needed, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Sliding window tracking the most frequent character in the window.

Valid window: window_size - max_freq <= k (replace the rest).

When invalid, shrink left by 1 — but NEVER decrease max_freq.

We only care about finding a LONGER valid window, so we only update max_freq upward.

This means the window never shrinks below the current best — it slides as a fixed size

until a better window is found. Time: O(n). Space: O(26)=O(1).

PHASE 4 CLEAN CODE

```
class _424_LongestRepeatingReplacement:
    def characterReplacement(self, s: str, k: int) -> int:
        freq     = [0] * 26
        max_freq = 0
        left     = 0
        longest  = 0
        for right in range(len(s)):
            freq[ord(s[right]) - ord('A')] += 1
            max_freq = max(max_freq, freq[ord(s[right]) -
ord('A')])
            if (right - left + 1) - max_freq > k:   # too
many replacements needed
                freq[ord(s[left]) - ord('A')] -= 1
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 TEST & DEBUG

"Let me trace characterReplacement('AABABBA', k=1)."

Sliding window. max_count=0 (count of most frequent char in window). left=0.

right=0(A): count={A:1}. max_count=1. window=1. 1-1=0<=k=1 ok. max_len=1.

right=1(A): count={A:2}. max_count=2. window=2. 2-2=0<=1 ok. max_len=2.

PHASE 5 TEST & DEBUG

right=2(B): count={A:2,B:1}. max_count=2. window=3. 3-2=1<=1 ok. max_len=3.

right=3(A): count={A:3,B:1}. max_count=3. window=4. 4-3=1<=1 ok. max_len=4.

right=4(B): count={A:3,B:2}. max_count=3. window=5. 5-3=2>1 -> shrink. remove s[0]='A'. count={A:2,B:2}. left=1. window=4. 4-3=1 ok... actually max_count doesn't decrease. Window size stays 4. max_len=4.

Continue: max_len=4 ok ('AABA' or 'ABAB' with one replacement).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**

'Don't decrease max_freq' is the key insight: we never shrink below the best window seen.

The window slides forward at a fixed size until max_freq improves, then grows.

Given more time I'd ask: unlimited replacements (k=n)? Answer = len(s).

MEDIUM | Sliding Window | LeetCode #438 | 7 of 21 | 7/21

Find All Anagrams in a String

QUESTION

Given two strings s and p, return an array of all the start indices of p's anagrams in s. You may return the answer in any order.

Example 1

Input	`s = "cbaebabacd", p = "abc"`
Output	`[0,6]`
Explanation	*The substring with start index = 0 is "cba", which is an anagram of "abc". The substring with start index = 6 is "bac", which is an anagram of "abc".*

Example 2

Input	`s = "abab", p = "ab"`
Output	`[0,1,2]`
Explanation	*The substring with start index = 0 is "ab", which is an anagram of "ab". The substring with start index = 1 is "ba", which is an anagram of "ab". The substring with start index = 2 is "ab", which is an anagram of "ab".*

CONSTRAINTS

- 1 <= s.length, p.length <= $3 * 10^4$
- s and p consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return all start indices in s where a substring is an anagram of p.

Fixed window size = len(p). Lowercase letters only. Right?"

"s='cbaebabacd', p='abc' -> [0,6] ok"

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for window in windows:
    # of length m in s
    sort the window
    if it equals sorted(p):
        record the start
```

TIME	SPACE
O(n * m log m)	O(m)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Sorts every window. A sliding window with a 26-letter count array updated as one letter enters and one leaves is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Fixed-size sliding window with frequency comparison — O(n). Walk-through:"

`p_count = frequency map of p. window_count = frequency map of first len(p) chars of s.`

Compare p_count == window_count -> if equal: add index 0 to result.

Slide window right by 1:

Add s[right] to window_count.

Remove s[right - len(p)] from window_count (element leaving window).

Compare again.

WHY compare whole dicts is O(1): dict size bounded by 26 chars -> fixed constant.

Alternatively track 'matches' counter — increment when freq[c] reaches p_count[c],

decrement when it leaves -> O(1) per step, avoid full dict comparison.

Time: O(n). Space: O(1) (26-char alphabet).

PHASE 4 CLEAN CODE

```python
class _438_FindAnagrams:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        if len(p) > len(s):
            return []
        p_count = Counter(p)
        w_count = Counter(s[:len(p)])
        result  = [0] if w_count == p_count else []
        for i in range(len(p), len(s)):
            w_count[s[i]] += 1
            left = s[i - len(p)]
            w_count[left] -= 1
            if w_count[left] == 0:
                del w_count[left]
            if w_count == p_count:
                result.append(i - len(p) + 1)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace findAnagrams('cbaebabacd','abc')."

p_count={a:1,b:1,c:1}. window of size 3. result=[].

window [0..2]='cba': count={c:1,b:1,a:1}==p_count -> append 0.

Slide: remove 'c', add 'e'. window='bae': {b:1,a:1,e:1}!=p_count.

Remove 'b', add 'b'. window='aeb': wait idx [2..4]='aeb'? No: 'cbaebabacd'[1:4]='bae'.

Systematic: i=0->'cba'=anagram->[0]. i=1->'bae'->no. i=2->'aeb'->no. i=3->'eba'->no. i=4->'bab'->no. i=5->'aba'->no. i=6->'bac'=anagram->[6].

Return [0,6] ok

PHASE 5 TEST & DEBUG

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** Counter comparison is O(26)=O(1) for fixed alphabet

Alternative: track a 'matches' counter instead of full compare — same O(n).

Given more time I'd ask: what if p can contain any Unicode?

Same algorithm — Counter handles any hashable.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #487 | 8 of 21 | 8/21

Max Consecutive Ones II

QUESTION

Given a binary array nums, return the maximum number of consecutive 1's in the array if you can flip at most one 0.

Example 1

Input	`nums = [1,0,1,1,0]`
Output	`4`
Explanation	*- If we flip the first zero, nums becomes [1,1,1,1,0] and we have 4 consecutive ones. - If we flip the second zero, nums becomes [1,0,1,1,1] and we have 3 consecutive ones. The max number of consecutive ones is 4.*

Example 2

Input	`nums = [1,0,1,1,0,1]`
Output	`4`
Explanation	*- If we flip the first zero, nums becomes [1,1,1,1,0,1] and we have 4 consecutive ones. - If we flip the second zero, nums becomes [1,0,1,1,1,1] and we have 4 consecutive ones. The max number of consecutive ones is 4.*

CONSTRAINTS

- 1 <= nums.length <= 10^5
- nums[i] is either 0 or 1.
- Follow up: What if the input numbers come in one by one as an infinite stream? In other words, you can't store all numbers coming from the stream as it's too large to hold in memory. Could you solve it efficiently?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Binary array. Flip at most one 0 to 1. Return the longest consecutive 1s run. Right?

Follow-up: infinite stream — can't store all numbers."

"[1,0,1,1,0] -> flip first 0 -> 4 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for zero in zeros:
    pretend it is a one and scan for the longest
        run of ones
    also try no flip
    best = max(best, candidate)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A full scan for every zero. A sliding window that contains at most one zero (shrink from the left when a second zero appears) is O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window allowing at most 1 zero flip — O(n). Walk-through:"

zeros=0, left=0, best=0.

For right in 0..n-1:

If nums[right]==0: zeros++.

While zeros > 1:

If nums[left]==0: zeros--.

left++.

`best = max(best, right-left+1).`

WHY zeros counter instead of tracking position: we only care how many zeros are in the window,

not where they are. If zeros > 1, shrink from left until back to <= 1 zero.

Variant: if unlimited flips (k flips), replace 1 with k — identical logic.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _487_MaxConsecutiveOnesII:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> i
nt:
        zeros   = 0
        left    = 0
        longest = 0
        for right in range(len(nums)):
            if nums[right] == 0:
                zeros += 1
            while zeros > 1:
                if nums[left] == 0:
                    zeros -= 1
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 TEST & DEBUG

"Let me trace findMaxConsecutiveOnesII([1,0,1,1,0]) — flip at most 1 zero."

Sliding window. Track last zero position.

left=0, last_zero=-1, max_len=0.

right=0(1): no zero. window=[0..0], len=1.

right=1(0): last_zero=1. window=[0..1], len=2.

right=2(1): no zero. window=[0..2], len=3.

PHASE 5 TEST & DEBUG

right=3(1): no zero. window=[0..3], len=4.

right=4(0): second zero! left moves to last_zero+1=2. last_zero=4. window=[2..4], len=3.

max_len=4 ok (window [0..3]=[1,0,1,1] with 0 flipped).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Stream follow-up: keep a deque of the last two 0-positions.

When third 0 arrives, left = oldest_zero_pos + 1 — O(1) space regardless of stream length.

Generalises to at-most-k flips (Max Consecutive Ones III #1004): same window, k zeros allowed.

Name that connection unprompted.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #567 | 9 of 21 | 9/21

Permutation in String

QUESTION

Given two strings s1 and s2, return true if s2 contains a permutation of s1, or false otherwise.

In other words, return true if one of s1's permutations is the substring of s2.

Example 1	
Input	`s1 = "ab", s2 = "eidbaooo"`
Output	`true`
Explanation	*s2 contains one permutation of s1 ("ba").*

Example 2	
Input	`s1 = "ab", s2 = "eidboaoo"`
Output	`false`

CONSTRAINTS

- 1 <= s1.length, s2.length <= 10^4
- s1 and s2 consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Does s2 contain a contiguous substring that is a permutation of s1 (i.e. an anagram)? Right?"

"A few quick questions:

Case sensitive, lowercase letters only per constraints? Yes."

"Let me walk: s1='ab', s2='eidbaooo' -> 'ba' at index 3 is a permutation of 'ab' -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for window in windows:
    # of length |s1| in s2
    sort the window
    if it equals sorted(s1):
        return True
```

TIME	SPACE
O(n * m log m)	O(m)

WHY IT IS SLOW

Sorts every window. A sliding window with a 26-letter count array, updated as one letter enters and one leaves, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is sorting each window — O(n*k log k).

Fixed-size sliding window with character frequency counts. Maintain a 'matches' counter of how

many of the 26 letters have matching counts between window and s1. Slide one char at a time,

updating counts and matches in O(1) per step.

Time: O(n). Space: O(26) = O(1)."

Key insight: a "matches" counter tracks how many of 26 letters have equal frequency — each character slide updates the counter in O(1), making the full sweep O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _567_PermutationInString:
    def checkInclusion(self, s1, s2):
        if len(s1) > len(s2):
            return False
        need = [0] * 26
        window = [0] * 26
        for ch in s1:
            need[ord(ch) - 97] += 1
        k = len(s1)
        for i in range(k):
            window[ord(s2[i]) - 97] += 1
        if window == need:
            return True
        for i in range(k, len(s2)):
            window[ord(s2[i]) - 97] += 1
            window[ord(s2[i - k]) - 97] -= 1
            if window == need:
                return True
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace checkInclusion('ab','eidbaooo')."

need={a:1,b:1}. window size=2.

[0..1]='ei': {e:1,i:1}!=need. Slide right.

[1..2]='id': no. [2..3]='db': no. [3..4]='ba': {b:1,a:1}==need -> return True ok

'ab' in 'eidboaoo': check each window of size 2.

Short circuit: track have/need count. have=0 (chars meeting required freq).

When have==len(need): window contains permutation ok

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** (26-letter arrays)

Follow-up: Find All Anagrams in a String (#438) — same technique, collect all start indices.

Follow-up: Minimum Window Substring (#76) — variable-size window instead of fixed.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #713 | 10 of 21 | 10/21

Subarray Product Less Than K

QUESTION

Given an array of integers nums and an integer k, return the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than k.

Example 1

Input	`nums = [10,5,2,6], k = 100`
Output	`8`
Explanation	*The 8 subarrays that have product less than 100 are: [10], [5], [2], [6], [10, 5], [5, 2], [2, 6], [5, 2, 6] Note that [10, 5, 2] is not included as the product of 100 is not strictly less than k.*

Example 2

Input	`nums = [1,2,3], k = 0`
Output	`0`

CONSTRAINTS

- 1 <= nums.length <= $3 * 10^4$
- 1 <= nums[i] <= 1000
- 0 <= k <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count subarrays where the product of all elements is strictly less than k. Right?"

"A few quick questions:

All values are positive integers? Yes — required for sliding window.

k can be <= 1? Yes — if k<=1 with all positive values, count=0."

"Let me walk: nums=[10,5,2,6], k=100 -> [10],[5],[2],[6],[10,5],[5,2],[2,6],[5,2,6] -> 8 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    extend the end while the running product is
        less than k
    count each valid end
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Restarts the product from every start. A sliding window that divides out nums[left] when the product reaches k, adding (right - left + 1) per step, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the $O(n^2)$ nested scan.

Sliding window: maintain window product. Expand right; if product >= k, shrink from left.

At each right position, all subarrays ending at right with left in [left..right] are valid.

Count += right - left + 1.

Time: O(n). Space: O(1)."

Key insight: for each right endpoint, all subarrays ending there with product < k start somewhere in [left..right] — their count is exactly (right - left + 1), accumulated in one pass.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _713_SubarrayProductLessThanK:
    def numSubarrayProductLessThanK(self, nums, k):
        if k <= 1:
            return 0
        product = 1
        left = 0
        count = 0
        for right in range(len(nums)):
            product *= nums[right]
            while product >= k:
                product //= nums[left]
                left += 1
            count += right - left + 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

nums=[10,5,2,6], k=100:

r=0:prod=10<100->count+=1. r=1:prod=50<100->count+=2. r=2:prod=100>=100->shrink:prod=10,l=1. count+=2.

r=3:prod=60<100->count+=3. total=1+2+2+3=8 ok

k=1: return 0 immediately ok

Count += right-left+1 counts all valid subarrays ending at right

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element added/removed from window at most once	**O(1)**

If values can be 0 or negative, sliding window doesn't work — need different approach.

Follow-up: Number of Subarrays with Product Less Than K II — with negatives; harder.

Follow-up: Maximum Product Subarray (#152) — max instead of count.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #837 | 11 of 21 | 11/21

New 21 Game

QUESTION

Alice plays the following game, loosely based on the card game "21".

Alice starts with 0 points and draws numbers while she has less than k points. During each draw, she gains an integer number of points randomly from the range [1, maxPts], where maxPts is an integer. Each draw is independent and the outcomes have equal probabilities.

Alice stops drawing numbers when she gets k or more points.

Return the probability that Alice has n or fewer points.

Answers within 10^{-5} of the actual answer are considered accepted.

Example 1

Input	`n = 10, k = 1, maxPts = 10`
Output	`1.00000`
Explanation	*Alice gets a single card, then stops.*

Example 2

Input	`n = 6, k = 1, maxPts = 10`
Output	`0.60000`
Explanation	*Alice gets a single card, then stops.* *In 6 out of 10 possibilities, she is at or below 6 points.*

Example 3

Input	`n = 21, k = 17, maxPts = 10`
Output	`0.73278`

CONSTRAINTS

- $0 <= k <= n <= 10^4$
- $1 <= maxPts <= 10^4$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Start with 0 points. Draw [1..maxPts] randomly until score >= n. Probability of score <= maxPts+n-1? Right?"

"A few quick questions:

We stop WHEN we reach >= n, and we want P(final score <= n+k-1)? Yes."

"Let me walk: n=1, k=10, maxPts=10 -> probability=1.0 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
dp[i] = the sum of dp[i - w] for w = 1..maxPts,
    divided by maxPts (while i - w is below k)
sum dp[k..n]
```

TIME	SPACE
O(n * maxPts)	O(n)

WHY IT IS SLOW

Re-adds up to maxPts probabilities for every point. Keep a sliding window sum (add the new term, subtract the one that leaves): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window DP on probability — O(n + W). Walk-through:"

`dp[x] = probability that total points == x after some draws.`

Initialize dp[0] = 1.0. Window sum = dp[0] = 1.0.

For x in 1..n+max_pts:

`dp[x] = window_sum / max_pts <- uniform over last max_pts draws.`

If x < n: window_sum += dp[x] (add new state to window if we haven't stopped yet).

If x >= max_pts: window_sum -= dp[x - max_pts] (remove state leaving window).

If x >= n: answer += dp[x]. <- x >= n means we stopped.

WHY sliding window: each dp[x] is the average of dp[x-W]..dp[x-1].

Maintaining a running sum avoids O(W) per state -> O(1) per state.

Time: O(n + W). Space: O(n + W).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _837_New21Game:
    def new21Game(self, n, k, maxPts):
        if k == 0 or n >= k + maxPts:
            return 1.0
        dp = [0.0] * (n + 1)
        dp[0] = 1.0
        window_sum = 1.0
        result = 0.0
        for i in range(1, n + 1):
            dp[i] = window_sum / maxPts
            if i < k:
                window_sum += dp[i]
            else:
                result += dp[i]
            if i >= maxPts:
                window_sum -= dp[i - maxPts]
```

```
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace new21Game(n=10,k=3,maxPts=2) — draw until score>=k, stay <=n."

dp[x]=prob of landing exactly at score x. dp[0]=1.0.

Window sum W=dp[0]=1. For x=1..n+maxPts:

x=1: dp[1]=W/maxPts=1/2=0.5. Update W+=dp[1].

x=2: dp[2]=W/2=(1+0.5)/2=0.75. W+=(0.75).

x=3 (=k): dp[3]=W/2. W+=dp[3]. But now if x>=k, subtract dp[x-maxPts=1]=dp[1] from W.

x=4 (>=k): dp[4]=W/2. W+=dp[4]. subtract dp[2].

x=5: dp[5]=W/2. subtract dp[3]. ... Stop at x=n=10.

Sum dp[k..n] = answer. Return prob ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+k)	**O(n)**

Follow-up: Soup Servings (#808) — similar probability DP.

Follow-up: if maxPts can be very large, different approach needed.

MEDIUM | Sliding Window | LeetCode #904 | 12 of 21 | 12/21

Fruit Into Baskets

QUESTION

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array fruits where fruits[i] is the type of fruit the ith tree produces.

You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

CONSTRAINTS

- You only have two baskets, and each basket can only hold a single type of fruit. There is no limit on the amount of fruit each basket can hold.
- Starting from any tree of your choice, you must pick exactly one fruit from every tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.
- Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array fruits, return the maximum number of fruits you can pick.

Example 1

Input	`fruits = [1,2,1]`
Output	`3`
Explanation	*We can pick from all 3 trees.*

Example 2

Input	`fruits = [0,1,2,2]`
Output	`3`
Explanation	*We can pick from trees [1,2,2]. If we had started at the first tree, we would only pick from trees [0,1].*

Example 3

Input	`fruits = [1,2,3,2,2]`
Output	`4`
Explanation	*We can pick from trees [2,3,2,2]. If we had started at the first tree, we would only pick from trees [1,2].*

- 1 <= fruits.length <= 10^5
- 0 <= fruits[i] < fruits.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We can pick fruits from at most 2 different types of trees consecutively.

Find the maximum number of fruits we can collect in one continuous segment. Right?"

"A few quick questions:

This reduces to: longest subarray with at most 2 distinct values? Yes."

"Let me walk: fruits=[1,2,1] -> collect all 3 (types 1,2) ok. fruits=[0,1,2,2] -> [1,2,2] len=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    extend j while the fruits seen are at most
        two types
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Restarts the window from every position. A sliding window with a count map of at most two types, shrinking from the left when a third appears, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window with at most 2 distinct fruit types — O(n). Walk-through:"

`basket = {} (fruit_type -> count). left=0. best=0.`

For right in 0..n-1:

`basket[fruits[right]] = basket.get(fruits[right],0)+1.`

While len(basket) > 2:

`basket[fruits[left]] -= 1.`

If basket[fruits[left]]==0: del basket[fruits[left]].

left++.

`best = max(best, right-left+1).`

WHY at most 2 types: problem = pick any subarray with <= 2 distinct values.

WHY sliding window: shrink from left when too many types.

Identical to Longest Substring with At Most 2 Distinct Characters (Q159).

Time: O(n). Space: O(1) (basket has <= 3 keys at once).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _904_FruitIntoBaskets:
    def totalFruit(self, fruits):
        basket = defaultdict(int)
        left = 0
        best = 0
        for right in range(len(fruits)):
            basket[fruits[right]] += 1
            while len(basket) > 2:
                basket[fruits[left]] -= 1
                if basket[fruits[left]] == 0:
                    del basket[fruits[left]]
                left += 1
            best = max(best, right - left + 1)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

fruits=[1,2,1]: basket={1:1}->{1:1,2:1}->{1:2,2:1}. len<=2 always. best=3 ok

fruits=[0,1,2,2]: r=2(2): basket={0:1,1:1,2:1} len=3->shrink: remove 0,l=1. basket={1:1,2:1}.

r=3(2): basket={1:1,2:2}. best=3 ok

Deleting zero-count entries keeps len(basket) accurate

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
each element added/removed once	counter has <= 3 entries

This is 'Longest Substring with At Most K Distinct Characters' (#340) with k=2.

Follow-up: extend to k baskets — same window, just change the limit to k.

Follow-up: if order matters (must pick from adjacent trees), sliding window still applies.

MEDIUM | Sliding Window | LeetCode #978 | 13 of 21 | 13/21

Longest Turbulent Subarray

QUESTION

Given an integer array arr, return the length of a maximum size turbulent subarray of arr.

A subarray is turbulent if the comparison sign flips between each adjacent pair of elements in the subarray.

More formally, a subarray [arr[i], arr[i + 1], ..., arr[j]] of arr is said to be turbulent if and only if:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A turbulent subarray alternates between comparisons: a>b<c or a<b>c.

Find the maximum length turbulent subarray. Right?"

"A few quick questions:

Single element is turbulent (length 1)? Yes. Two equal adjacent elements break turbulence? Yes."

"Let me walk: arr=[9,4,2,10,7,8,8,1,9] -> [4,2,10,7,8] length 5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    extend while the comparison signs keep
        alternating
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Restarts from every position. One pass tracking the length of the current alternating run (reset to 2 or 1 when the pattern breaks) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Kadane's on turbulent subarray length — O(n). Walk-through:"

inc = dec = 1 (length of current turbulent subarray ending with up or down move).

For i in 1..n-1:

If arr[i] > arr[i-1]: inc = dec+1; dec=1. <- current step is up; extends a subarray that ended going down

Elif arr[i] < arr[i-1]: dec = inc+1; inc=1. <- current step is down; extends a subarray that ended going up

PHASE 3 OPTIMIZE

Else: inc = dec = 1. <- equal -> turbulence breaks; reset

best = max(best, inc, dec).

WHY track inc and dec separately: turbulence requires alternating. An upward step can only

extend a subarray that ended going down, and vice versa.

WHY reset both to 1 on equal: equal consecutive elements kill turbulence.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _978_LongestTurbulentSubarray:
    def maxTurbulenceSize(self, arr):
        n = len(arr)
        if n < 2:
            return n
        best = cur = 1
        for i in range(1, n):
            c = (arr[i] > arr[i-1]) - (arr[i] < arr[i-1])
            if i == 1 or c == 0 or c == ((arr[i-1] > arr[i
-2]) - (arr[i-1] < arr[i-2])):
                cur = 1 if c == 0 else 2
            else:
                cur += 1
            best = max(best, cur)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

arr=[9,4,2,10,7,8,8,1,9]:

i=1(4<9,c=-1): cur=2. i=2(2<4,c=-1): same sign->reset cur=2. i=3(10>2,c=+1): alt->cur=3.

i=4(7<10,c=-1): alt->cur=4. i=5(8>7,c=+1): alt->cur=5. i=6(8==8,c=0): reset->cur=1.

i=7(1<8,c=-1): cur=2. i=8(9>1,c=+1): alt->cur=3. best=5 ok

C==0 (equal elements) always resets; same-sign sequence also resets

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: count all turbulent subarrays — sum of lengths is trickier.

Follow-up: Wiggle Sort (#280) — rearrange array to be turbulent.

MEDIUM | Sliding Window | LeetCode #1004 | 14 of 21 | 14/21

Max Consecutive Ones III

QUESTION

Given a binary array nums and an integer k, return the maximum number of consecutive 1's in the array if you can flip at most k 0's.

Example 1

Input	`nums = [1,1,1,0,0,0,1,1,1,1,0], k = 2`
Output	`6`
Explanation	*[1,1,1,0,0, 1 ,1,1,1,1, 1] Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.*

Example 2

Input	`nums = [0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1], k = 3`
Output	`10`
Explanation	*[0,0, 1,1, 1 , 1 ,1,1,1, 1 ,1,1 ,0,0,0,1,1,1,1] Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.*

CONSTRAINTS

- 1 <= nums.length <= 10^5
- nums[i] is either 0 or 1.
- 0 <= k <= nums.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Binary array; can flip at most k zeros to ones. Return the max length consecutive 1s run. Right?"

"A few quick questions:

k=0 means no flips allowed? Yes — find max existing 1s run.

The array is not modified — we're just counting? Yes."

"Let me walk: nums=[1,1,1,0,0,0,1,1,1,1,0], k=2 -> flip two 0s -> max run = 6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subarray in subarrays:
    count its zeros
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Visits every subarray. A sliding window that shrinks from the left when it contains more than k zeros is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window: allow at most k zeros — O(n). Walk-through:"

zeros=0, left=0, best=0.

For right in 0..n-1:

If nums[right]==0: zeros++.

While zeros > k: if nums[left]==0: zeros--; left++.

`best = max(best, right-left+1).`

WHY at most k zeros: flipping k zeros means the window can have up to k zeros.

Once zeros > k, shrink from left until we have <= k zeros again.

WHY sliding window: all elements are 0 or 1; valid windows are contiguous; expanding right

never makes a valid window invalid (only adding 0s changes validity).

Identical logic to Q487 (flip at most 1 zero) generalized to k.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1004_MaxConsecutiveOnesIII:
    def longestOnes(self, nums, k):
        left = 0
        zero_count = 0
        best = 0
        for right in range(len(nums)):
            if nums[right] == 0:
                zero_count += 1
            while zero_count > k:
                if nums[left] == 0:
                    zero_count -= 1
                left += 1
            best = max(best, right - left + 1)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

nums=[1,1,1,0,0,0,1,1,1,1,0], k=2:

Expand until r=4: zeros=2<=2. r=5: zeros=3>2->shrink until zero_count=2 (l=3).

Continue expanding. Best window=[0,0,1,1,1,1] at r=10? Best=6 ok

k=0: no zeros allowed -> max run of existing 1s ok

Shrinking stops as soon as zero_count returns to k

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element processed once	**O(1)**

Follow-up: Max Consecutive Ones II (#487) — k=1 (at most one flip); stream follow-up.

Follow-up: Max Consecutive Ones I (#485) — k=0 (no flips); track current run.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #1100 | 15 of 21 | 15/21

Find K-Length Substrings with No Repeated Characters

QUESTION

Given a string s and an integer k, return the number of substrings in s of length k with no repeated characters.

Example 1

Input	`s = "havefunonleetcode", k = 5`
Output	`6`
Explanation	*There are 6 substrings they are: 'havef','avefu','vefun','efuno','etcod','tcode'.*

Example 2

Input	`s = "home", k = 5`
Output	`0`
Explanation	*Notice k can be larger than the length of s. In this case, it is not possible to find any substring.*

CONSTRAINTS

- 1 <= s.length <= 10^4
- s consists of lowercase English letters.
- 1 <= k <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count substrings of exactly length k with all distinct characters. Right?

k > len(s) -> 0. k=0 -> 0."

"s='havefunonleetcode', k=5 -> 6 ok"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    take the k-length substring and check whether
        its letters are all different
    count the ones that are
```

TIME	SPACE
O(n * k)	O(k)

WHY IT IS SLOW

Rebuilds and checks each window from scratch. Slide a window with a count of letters (or a last-seen position) updated by one letter per step: O(n).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Fixed-size sliding window with distinct char count — O(n). Walk-through:"

Use a freq map and 'distinct' counter.

Build initial window of size k: freq each char, count distinct.

If distinct == k (all chars unique): count++.

Slide:

Remove leftmost char: freq[s[left]]--; if 0: distinct--; left++.

Add rightmost char: if freq[s[right]]==0: distinct++; freq[s[right]]++; right++.

If distinct == k: count++.

WHY distinct == k for uniqueness: k distinct chars in a window of size k -> all unique.

WHY fixed window: problem specifies exactly k chars.

Time: O(n). Space: O(k).

PHASE 4 CLEAN CODE

```
class _1100_KLengthSubstrings:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> in
t:
        if k > len(s):
            return 0
        freq  : defaultdict = defaultdict(int)
        count = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            if right >= k:
                left = right - k
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
            if right >= k - 1 and len(freq) == k:
                count += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace numKLenSubstrNoRepeats('havefunonleetcode', k=5)."

Sliding window of size k=5, check for repeats using set/counter

Window 'havef': set={h,a,v,e,f}=5 unique. No repeat -> count=1

Slide: remove 'h', add 'u': 'avefu': 5 unique -> count=2

Slide: remove 'a', add 'n': 'vefun': 5 unique -> count=3

Continue... 'eetco': 'e' repeats -> count stays

Return total count of windows with all unique chars ok

WHY fixed-size sliding window + set: O(n) amortized; add/remove chars as window slides.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(k)** len(freq)==k is the clean validity check

Fixed window variant of #3 — useful when window size is given, not maximised.

Given more time I'd ask: return the actual substrings? Collect s[right-k+1:right+1].

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #1248 | 16 of 21 | 16/21

Count Number of Nice Subarrays

QUESTION

Given an array of integers nums and an integer k. A continuous subarray is called nice if there are k odd numbers on it.

Return the number of nice sub-arrays.

Example 1	
Input	`nums = [1,1,2,1,1], k = 3`
Output	`2`
Explanation	*The only sub-arrays with 3 odd numbers are [1,1,2,1] and [1,2,1,1].*

Example 2	
Input	`nums = [2,4,6], k = 1`
Output	`0`
Explanation	*There are no odd numbers in the array.*

Example 3	
Input	`nums = [2,2,2,1,2,2,1,2,2,2], k = 2`
Output	`16`

CONSTRAINTS

- 1 <= nums.length <= 50000
- 1 <= nums[i] <= 10^5
- 1 <= k <= nums.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Count subarrays with EXACTLY k odd numbers. Return the count. Right?

Map each element to 1 (odd) or 0 (even) -> reduce to Subarray Sum Equals K (#560)."

"[1,1,2,1,1], k=3 -> 2 ok. [2,2,2,1,2,2,1,2,2,2], k=2 -> 16 ok."

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subarray in subarrays:
    count its odd numbers
    count those with exactly k
```

TIME	SPACE
O(n^2)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Visits every subarray. Count prefixes by the number of odds so far in a table: each position adds the count of prefixes with (odds - k): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window on odd-count subarrays — O(n). Walk-through:"

Count subarrays with exactly k odd numbers = count(at most k) - count(at most k-1).

`at_most(k): sliding window counting subarrays with <= k odds.`

odds=0, left=0, count=0.

For right in 0..n-1:

If nums[right]%2==1: odds++.

While odds > k: if nums[left]%2==1: odds--; left++.

`count += right-left+1 (all subarrays ending at right with left..right start).`

`return count.`

Answer = at_most(k) - at_most(k-1).

WHY this difference: subarrays with exactly k odds = those with <=k minus those with <=k-1.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _1248_NiceSubarrays:
    def numberOfSubarrays(self, nums: List[int], k: int) -
> int:
        prefix_count = Counter({0: 1})
        odd_sum      = 0
        count        = 0
        for num in nums:
            odd_sum += num % 2
            count   += prefix_count[odd_sum - k]
            prefix_count[odd_sum] += 1
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace numberOfSubarrays([1,1,2,1,1], k=3)."

Count subarrays with exactly k odd numbers. = atMost(k) - atMost(k-1).

`atMost(3): sliding window counting odd numbers.`

Prefix sum: prefix[i]=count of odds in arr[0..i-1]. seen={0:1}.

i=0(1,odd): prefix=1. seen[1-3]=seen[-2]=0. count+=0. seen={0:1,1:1}.

i=1(1,odd): prefix=2. seen[-1]=0. seen={0:1,1:1,2:1}.

i=2(2,even): prefix=2. seen[2-3]=seen[-1]=0. seen={...}.

i=3(1,odd): prefix=3. seen[0]=1. count+=1.

i=4(1,odd): prefix=4. seen[1]=1. count+=1.

PHASE 5 TEST & DEBUG

Total=2 ok ([1,1,2,1] and [1,2,1,1] subarrays with 3 odds? Let me verify: [1,1,2,1] has odds at 0,1,3 -> 3 odds ok. [1,2,1,1] has odds at 0,2,3 -> 3 ok. Return 2 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** Identical to #560 — name the connection explicitly

At-most-k version: sliding window counting odds in window, shrink when > k.

Exactly k = atMost(k) - atMost(k-1).

Given more time I'd ask: count subarrays with at most k odd numbers?

Sliding window — shrink when odd count exceeds k, add right-left+1 each step.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #1423 | 17 of 21 | 17/21

Maximum Points You Can Obtain from Cards

QUESTION

There are several cards arranged in a row, and each card has an associated number of points. The points are given in the integer array cardPoints.

In one step, you can take one card from the beginning or from the end of the row. You have to take exactly k cards.

Your score is the sum of the points of the cards you have taken.

Given the integer array cardPoints and the integer k, return the maximum score you can obtain.

Example 1

Input	`cardPoints = [1,2,3,4,5,6,1], k = 3`
Output	`12`
Explanation	*After the first step, your score will always be 1. However, choosing the rightmost card first will maximize your total score. The optimal strategy is to take the three cards on the right, giving a final score of 1 + 6 + 5 = 12.*

Example 2

Input	`cardPoints = [2,2,2], k = 2`
Output	`4`
Explanation	*Regardless of which two cards you take, your score will always be 4.*

Example 3

Input	`cardPoints = [9,7,7,9,7,7,9], k = 7`
Output	`55`
Explanation	*You have to take all the cards. Your score is the sum of points of all cards.*

CONSTRAINTS

- 1 <= cardPoints.length <= 10^5
- 1 <= cardPoints[i] <= 10^4
- 1 <= k <= cardPoints.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Pick k cards from either the left end or right end of the row. Maximise total points. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Must pick exactly k cards? Yes. Can mix left and right picks? Yes."

"Let me walk: cardPoints=[1,2,3,4,5,6,1], k=3 -> pick [6,1] from right and [1] from left? No.

Best: take last 3 [5,6,1]=12 or [1]+[5,6]=12 or [1,2]+[1]=4. -> max=12 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(0,_k_+_1):
    take the first i cards and the last k - i
        cards and add them up
    best = max(best, candidate)
```

TIME	SPACE
O(k^2)	O(1)

WHY IT IS SLOW

Re-adds up to k cards for every split. The total of the first i plus last k - i changes by one card each step, so a sliding sum (or prefix and suffix sums) is O(k).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is recomputing the sum for each combination.

Equivalent insight: minimize the sum of the (n-k) middle elements (the cards we DON'T pick).

Sliding window of size n-k on the remaining array.

Or: sliding window of left+right picks.

Time: O(k) or O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1423_MaximumPointsFromCards:
    def maxScore(self, cardPoints, k):
        n = len(cardPoints)
        window_size = n - k
        if window_size == 0:
            return sum(cardPoints)
        window_sum = sum(cardPoints[:window_size])
        min_window = window_sum
        for i in range(window_size, n):
            window_sum += cardPoints[i] - cardPoints[i - w
indow_size]
            min_window = min(min_window, window_sum)
        return sum(cardPoints) - min_window
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

cardPoints=[1,2,3,4,5,6,1], k=3, n=7, window_size=4:

total=22. first window [1,2,3,4]=10. slide: [2,3,4,5]=14, [3,4,5,6]=18, [4,5,6,1]=16.

min_window=10. 22-10=12 ok

k=n: window_size=0 -> return total ok

The middle window to minimize = the cards we leave behind

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) one full scan	**O(1)**

The sliding-window-minimum insight is elegant: instead of maximising picked cards,

we minimise the unchosen middle segment.

Follow-up: if we could pick from any position (not just ends) — greedy max k cards.

Follow-up: Minimum Size Subarray Sum (#209) — different constraint, similar window idea.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #1456 | 18 of 21 | 18/21

Maximum Number of Vowels in a Substring of Given Length

QUESTION

Given a string s and an integer k, return the maximum number of vowel letters in any substring of s with length k.

Vowel letters in English are 'a', 'e', 'i', 'o', and 'u'.

Example 1	
Input	`s = "abciiidef", k = 3`
Output	3
Explanation	*The substring "iii" contains 3 vowel letters.*

Example 2	
Input	`s = "aeiou", k = 2`
Output	2
Explanation	*Any substring of length 2 contains 2 vowels.*

Example 3	
Input	`s = "leetcode", k = 3`
Output	2
Explanation	*"lee", "eet" and "ode" contain 2 vowels.*

CONSTRAINTS

- 1 <= s.length <= 10^5
- s consists of lowercase English letters.
- 1 <= k <= s.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the maximum number of vowels in any substring of length exactly k. Right?"

"A few quick questions:

Vowels are a,e,i,o,u? Yes. k <= len(s) guaranteed? Yes."

"Let me walk: s='abciiidef', k=3 -> 'iii' has 3 vowels -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for window in windows(length_k):
    count the vowels
    best = max(best, candidate)
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n * k)	O(1)

WHY IT IS SLOW

Recounts each window. Slide the window: add the entering letter's vowel flag and subtract the leaving one: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Fixed-size sliding window counting vowels — O(n). Walk-through:"

`vowels = set('aeiou').`

count vowels in first window s[0..k-1]. best = count.

For i in k..n-1:

If s[i] in vowels: count++. (add incoming char)

If s[i-k] in vowels: count--. (remove outgoing char)

`best = max(best, count).`

WHY O(n): each character enters and leaves the window exactly once.

Window size is fixed at k — no need to track left pointer explicitly (left = right - k).

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1456_MaxNumberVowelsInSubstring:
    def maxVowels(self, s, k):
        vowels = set('aeiou')
        count = sum(c in vowels for c in s[:k])
        max_count = count
        for i in range(k, len(s)):
            count += (s[i] in vowels) - (s[i-k] in vowels)
            max_count = max(max_count, count)
        return max_count
```

PHASE 5 TEST & DEBUG

"Let me trace maxVowels('abciiidef', k=3)."

vowels={a,e,i,o,u}. Compute first window 'abc': a->count=1, b->skip, c->skip. curr=1, max=1.

Slide i=3 (add 'i', remove 'a'): curr=1-0+1=1. max=1.

Slide i=4 (add 'i', remove 'b'): curr=1-0+1=2. max=2.

Slide i=5 (add 'i', remove 'c'): curr=2-0+1=3. max=3. <- peak

Slide i=6 (add 'd', remove 'i'): curr=3-1+0=2. max stays 3.

Slide i=7 (add 'e', remove 'i'): curr=2-1+1=2. max stays 3.

Slide i=8 (add 'f', remove 'i'): curr=2-1+0=1. max stays 3.

Return 3 ok (window 'iii' at positions 3-5)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) build first window O(k) + slide O(n-k)	**O(1)**

Follow-up: Maximum Average Subarray I (#643) — same fixed-window sliding pattern.

Follow-up: if k can change per query, sliding window needs to be recomputed each time.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #1838 | 19 of 21 | 19/21

Frequency of the Most Frequent Element

QUESTION

The frequency of an element is the number of times it occurs in an array.

You are given an integer array nums and an integer k. In one operation, you can choose an index of nums and increment the element at that index by 1.

Return the maximum possible frequency of an element after performing at most k operations.

Example 1

Input	`nums = [1,2,4], k = 5`
Output	3
Explanation	*Increment the first element three times and the second element two times to make nums = [4,4,4]. 4 has a frequency of 3.*

Example 2

Input	`nums = [1,4,8,13], k = 5`
Output	2
Explanation	*There are multiple optimal solutions: - Increment the first element three times to make nums = [4,4,8,13]. 4 has a frequency of 2. - Increment the second element four times to make nums = [1,8,8,13]. 8 has a frequency of 2. - Increment the third element five times to make nums = [1,4,13,13]. 13 has a frequency of 2.*

Example 3

Input	`nums = [3,9,6], k = 2`
Output	1

CONSTRAINTS

- $1 <= nums.length <= 10^5$
- $1 <= nums[i] <= 10^5$
- $1 <= k <= 10^5$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We can increment any element k total times. Find the maximum frequency any single value can achieve. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Each increment adds 1 to one element. Total increments across all operations <= k? Yes."

"Let me walk: nums=[1,2,4], k=5 -> make all 4: cost=3+2=5. frequency=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for value in values:
    # in nums
    add up the increments needed to raise the
        smaller elements to it
    keep the largest count whose total is at most k
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Recomputes the cost for each target. Sort, then slide a window where cost = nums[right] * window size - window sum; shrink while the cost exceeds k: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the $O(n^2)$ inner scan.

Sort nums. Sliding window: for a window [left..right], cost to make all equal to nums[right]

= nums[right]*(right-left+1) - window_sum.

If cost > k, shrink left; otherwise update best.

Time: O(n log n) for sort + O(n) window. Space: O(1)."

Key insight: after sorting, the cost to make window [left..right] all equal to nums[right] is nums[right]*(right-left+1) - window_sum — when cost > k, shrink from the left; (right - left + 1) at each step counts valid elements.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1838_FrequencyMostFrequentElement:
    def maxFrequency(self, nums, k):
        nums.sort()
        left = 0
        window_sum = 0
        best = 1
        for right in range(len(nums)):
            window_sum += nums[right]
            while nums[right] * (right - left + 1) - windo
w_sum > k:
                window_sum -= nums[left]
                left += 1
            best = max(best, right - left + 1)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

nums=[1,2,4], k=5 -> sorted=[1,2,4]:

r=0: sum=1, cost=0. r=1: sum=3, cost=2*2-3=1<=5. r=2: sum=7, cost=3*4-7=5<=5. best=3 ok

nums=[1,1,1,2,2,4], k=3 -> sorted: r=5(4): cost=6*4-11=13>3 -> shrink. Eventually best=3 ok

Nums[right]*(right-left+1)-window_sum = total increments needed for window

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) sort dominates	**O(1)**

The window sum trick avoids re-summing from scratch each step.

Follow-up: if we could also decrement, the answer is always n (make all equal to min).

Follow-up: Minimum Operations to Make Array Equal (#1551) — related frequency manipulation.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Sliding Window | LeetCode #1871 | 20 of 21 | 20/21

Jump Game VII

QUESTION

You are given a 0-indexed binary string s and two integers minJump and maxJump. In the beginning, you are standing at index 0, which is equal to '0'. You can move from index i to index j if the following conditions are fulfilled:

CONSTRAINTS

- i + minJump <= j <= min(i + maxJump, s.length - 1), and
- s[j] == '0'.

Return true if you can reach index s.length - 1 in s, or false otherwise.

Example 1

Input	`s = "0 11 0 1 0 ", minJump = 2, maxJump = 3`
Output	`true`
Explanation	*In the first step, move from index 0 to index 3. In the second step, move from index 3 to index 5.*

Example 2

Input	`s = "01101110", minJump = 2, maxJump = 3`
Output	`false`

- 2 <= s.length <= 10^5
- s[i] is either '0' or '1'.
- s[0] == '0'
- 1 <= minJump <= maxJump < s.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Binary string. Can jump from index i to i+1 or i+2 if the target character == s[i].

Can we reach the last index? Right?"

"A few quick questions:

Start at index 0. s[0] and s[last] are guaranteed to be '0'? Yes per constraints."

"Let me walk: s='011010', k=2 -> reachable? From 0(0), jump to 1 or 2. s[1]='1',s[2]='1'. But need s[i]=='0'. -> False? Actually can jump: from 0 reach positions where s==s[current]. Hmm—let me recheck the problem.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(i):
    for j in js:
        # in [i + minJump, i + maxJump] where
            s[j] == '0'
        if dfs(j):
            return True
    return i == n - 1
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Branches on every reachable index and repeats indices. A BFS or DP where each index is reached once, with a sliding count of reachable indices in the jump window, is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(n*k) inner loop.

Prefix sum of reachable: reach_sum[i] = number of reachable indices in [0..i].

For position j, we can reach it if any index in [j-maxJump, j-minJump] is reachable.

Check reach_sum[j-minJump] - reach_sum[j-maxJump-1] > 0.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1871_JumpGameVII:
    def canReach(self, s, minJump, maxJump):
        n = len(s)
        reachable = [False] * n
        reachable[0] = True
        prefix = [0] * (n + 1)
        prefix[1] = 1
        for j in range(1, n):
            if s[j] == '0':
                lo = max(0, j - maxJump)
                hi = j - minJump
                if hi >= 0 and prefix[hi + 1] - prefix[lo]
 > 0:
                    reachable[j] = True
            prefix[j + 1] = prefix[j] + (1 if reachable[j]
 else 0)
        return reachable[n - 1]
```

PHASE 5 TEST & DEBUG

"Let me trace canReach(s='011010', minJump=2, maxJump=3)."

BFS / greedy with prefix count of reachable positions

`reachable[i] = can we reach index i?`

reachable[0]=True (start). cnt=prefix_reachable_count

i=2: check if any j in [max(0,i-maxJump), i-minJump]=[0,0] is reachable. reachable[0]=True, s[2]='1' ok -> True

i=3: check [0,1]: reachable[0]=True but s[3]='0' -> False

i=4: check [1,2]: reachable[2]=True, s[4]='1' ok -> True

i=5: check [2,3]: reachable[2]=True, s[5]='0' -> False

Return reachable[n-1=5]=False ok? Wait s[5]='0'... check s='011010'

s[5]='0' -> return False ok (last char '0' can't land there per problem? No: last pos must be 0)

BFS/greedy: queue=[0] (index 0, start). visited/reachable[i]=True. Pop i: for j in [i+minJump..i+maxJump] where s[j]='1': if not visited -> enqueue. Mark visited to prevent revisit.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**

Follow-up: Jump Game VI (#1696) — maximise score; uses monotonic deque on DP.

Follow-up: Jump Game I/II (#55/#45) — jump from any position to any distance.

MEDIUM | Sliding Window | LeetCode #2461 | 21 of 21 | 21/21

Maximum Sum of Distinct Subarrays With Length K

QUESTION

You are given an integer array nums and an integer k. Find the maximum subarray sum of all the subarrays of nums that meet the following conditions:

CONSTRAINTS

- The length of the subarray is k, and
- All the elements of the subarray are distinct.

Return the maximum subarray sum of all the subarrays that meet the conditions. If no subarray meets the conditions, return 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1

Input	`nums = [1,5,4,2,9,9,9], k = 3`
Output	`15`
Explanation	*The subarrays of nums with length 3 are: - [1,5,4] which meets the requirements and has a sum of 10. - [5,4,2] which meets the requirements and has a sum of 11. - [4,2,9] which meets the requirements and has a sum of 15. - [2,9,9] which does not meet the requirements because the element 9 is repeated. - [9,9,9] which does not meet the requirements because the element 9 is repeated. We return 15 because it is the maximum subarray sum of all the subarrays that meet the conditions*

Example 2

Input	`nums = [4,4,4], k = 3`
Output	`0`
Explanation	*The subarrays of nums with length 3 are: - [4,4,4] which does not meet the requirements because the element 4 is repeated. We return 0 because no subarrays meet the conditions.*

- $1 <= k <= nums.length <= 10^5$
- $1 <= nums[i] <= 10^5$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the maximum sum of a subarray of length k where all elements are distinct. Right?"

"A few quick questions:

If no valid window (all windows have duplicates)? Return 0. Values can be large? Yes."

"Let me walk: nums=[1,5,4,2,9,9,9], k=3 -> [1,5,4]=10, [5,4,2]=11, [4,2,9]=15, [2,9,9] invalid, [9,9,9] invalid -> 15 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for window in windows(length_k):
    check that its elements are distinct (a set)
        and add them up
    best = max(best, valid_sum)
```

TIME	SPACE
O(n * k)	O(k)

WHY IT IS SLOW

Rebuilds each window. Slide the window with a count map and a running sum, tracking how many values occur more than once: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is O(k) set check and sum per window.

Sliding window with a frequency Counter: maintain current sum and distinct count.

Add new element; if it causes a duplicate, shrink from left until distinct again.

Since window is exactly k, just slide and track duplicates.

Time: O(n). Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _2461_MaxSumDistinctSubarrays:
    def maximumSubarraySum(self, nums, k):
        freq = defaultdict(int)
        window_sum = 0
        best = 0
        for i, num in enumerate(nums):
            freq[num] += 1
            window_sum += num
            if i >= k:
                left = nums[i - k]
                freq[left] -= 1
```

```
            if freq[left] == 0: del freq[left]
            window_sum -= left
        if i >= k - 1 and len(freq) == k:
            best = max(best, window_sum)
    return best
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

nums=[1,5,4,2,9,9,9], k=3:

i=2: window [1,5,4], freq={1:1,5:1,4:1}, len=3==k -> sum=10, best=10.

i=3: remove 1, add 2. window [5,4,2], len=3 -> sum=11, best=11.

i=4: remove 5, add 9. window [4,2,9], len=3 -> sum=15, best=15.

i=5: remove 4, add 9. freq={2:1,9:2}, len=2!=3 -> skip.

i=6: remove 2, add 9. freq={9:3}, len=1!=3 -> skip. -> 15 ok

Len(freq)==k is an O(1) distinct-count check

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(k)** for the frequency map

Follow-up: if window size can vary, use a dynamic sliding window with distinct count.

Follow-up: Maximum Sum of Subarrays with K Distinct (#904 variant) — similar freq tracking.